TODO:
Record

GENERICS

```java
// class
class Box<T> { }
// variable
Box<String> b = new Box<String>();
// method
public <T> void doStuff(T value) { }
// Full Example
class OrderedPair<T, U> {
  T first;
  U second;
  public OrderedPair(T t, U u) {
    this.first = f;
    this.second = s;
  }
  public T getFirst() { return first; }
  public U getSecond() { return second; }
  @Override
  public int hashCode() {
    return Objects.hash(first.hashCode(),
                        second.hashCode());
  }
  @Override
  public boolean equals(Object obj) {
      if (obj == null ||
          obj.getClass() != getClass()) return false;
      @SuppressWarnings("unchecked")
      var other = (OrderedPair<T, U>)obj;
      return Objects.equals(first, other.first) &&
             Objects.equals(second, other.second);
  }
}

public class Employee implements Comparable<Employee>
{
  // ...
  @Override
  public int compareTo(Employee other) {
    return Double.compare(this.salary, other.salary);
  }
}
```

JAVA BEING STUPID

```java
T obj = new T();             // error
obj instanceof T;            // error
T heh = (T) "asdf";          // no typechecking
var s = Stack<int>();        // no primitive types
private static T x;          // static no workey
Node<Number> s = new Node<Integer>; // error
class IThrowAnErrorBecauseWhyNot {
  void m(List<String> list) { }
  void m(List<Integer> list) { }
} // error because of type erasure & identifiability
```

ITERATOR

```java
for (String s : stringList) { } // ==
for (Iterator<String> i = stringList.iterator();
    i.hasNext()) String s = i.next();
```

ITERABLE

```java
interface Iterable<T> {
  Iterator<T> iterator();
}
interface Iterator<E> {
  boolean hasNext();
  E next();
}
```

COMPARABLE

TODO:
Type signature?

```java
static <T extends Comparable> T doStuff(T a, T b) {
  // compareTo is possible with all types
  return a.compareTo("b") > 0 ? a : b;
}
static <T extends Comparable<T>> T doStuff(T a,T b) {
  // compareTo is possible only with T
  return a.compareTo(b) > 0 ? a : b;
}
```

EXAMPLE

```java
class Graphic {
  public void draw() { }
}
class Rectangle extends Graphic { }
class Circle extends Graphic { }
class GraphicStack<T extends Graphic>
    extends Stack<T> {
  public void drawAll() {
    for (T item : this) item.draw();
  }
}
```

BYTECODE

TODO:

TYPE ERASURE

- Introduced because of "bAcKwArDs CoMpAtAbIlItY"
- No type information at runtime (Non-Reifiable Type)

```java
class MyStack<T> {
  Entry<T> top;
  int push(T value) { }
}
// becomes
class MyStack {
  Entry top;
  int push(Object value) { }
}
/* with bounds */
class MyStack<T extends Comparable<T>> {
  Entry<T> top;
  int push(T value) { }
}
// becomes
class MyStack {
  Entry top;
  int push(Comparable value) { }
}
/* what happens at/before runtime */
MyStack<String> s = new MyStack<String>();
s.push("Eh");
String x = s.pop();
// becomes
MyStack s = new MyStack();
s.push("Eh");
String x = (String) s.pop();
```

WILDCARDS

```java
void printGs(List<Graphic> gs) { }
List<Graphic> gs1 = new ArrayList<>();
printGs(gs1);   // ok
List<Circle> gs2 = new ArrayList<>();
printGs(gs2);   // error
/* solution */
void printGs(List<? extends Graphic> gs) { }
```

GENERIC VARIANCE

	Type	*Compatible Type-Args*	*R*	*W*
Invariance	`C<T>`	`T`	✓	✓
Covariance	`C<? extends T>`	`T` and sub-types	✓	✗
Contravariance	`C<? super T>`	`T` and basis-types	✗	✓
Bivariance	`C<?>`	All	✗	✗

INVARIANCE

```java
static <T> void move(Stack<T>, from, Stack<T> to) {
  while(!from.isEmpty()) to.push(from.pop());
}
var rectS = new Stack<Rectangle>();
var graphS = new Stack<Graphic>();
graphS.push(new Rectangle());        // ok
move(rectS, graphS);                 // error
Stack<Object> objS = graphS;         // error
/* anotherone */
String[] strA = new String[10];
Object[] objA = strA;                // ok
objA[0] = Integer.valueOf(2);        // runtime err
```

TODO:

```java
// compiles?
List<Graphic> list = new ArrayList<Rectangle>();
// compiles?
Rectangle[] rectangleStack = new Rectangle[10];
Graphic[] graphicStack = new Graphic[10];
graphicStack = rectangleStack;
// compiles?
Stack<Rectangle> rectangleStack = new Stack<>();
Stack<Graphic> graphicStack = new Stack<>();
graphicStack = rectangleStack;
// compiles?
Stack<Rectangle> rectangleStack = new Stack<>();
Stack<Graphic> graphicStack = new Stack<>();
for (Rectangle r : rectangleStack) {
  graphicStack.push(r);
}
```

COVARIANCE

```java
public class Stack<E> {
  public void pushAll(Iterable<? extends E> src) { }
}
Stack<? extends Graphic> graphS;
graphS = new Stack<Rectangle>();    // ok
graphS = new Stack<Circle>();       // ok
graphS = new Stack<Object>();       // error (duh)
/* read only */
graphS.push(new Graphic());          // error
graphS.push(new Rectangle());        // error
graphS.push(new Triangle());         // error
```

CONTRAVARIANCE

```java
static <T> void addToC(List<? super T> list, T e) {
  list.add(e);
}
addToC(new ArrayList<Number>(), 3); // ok
addToC(new ArrayList<Object>(), 3); // ok
/* shenanigans */
Stack<? super Graphic> s = new Stack<Object>();
s.add(new Graphic());               // ok
s.add(new Rectangle());             // ok
Graphic g = s.pop();                // error
Object g = s.pop();                 // ok
```

BIVARIANCE

```java
static void printList(List<?> list) {
  for (Object elem : list)
    System.out.print(elem + " ");
}
/* more shenanigans */
static void appendNewObject(List<?> list) {
  list.add(new Object());           // error
}
```

TODO:
more examples

ANOTHERONE

```java
public class VarianceExamples {
  public static double sum(
      Collection<? extends Number> numbers) {
    double sum = 0;
    for (Number num : numbers)
      sum += num.doubleValue();
    return sum;
  }
  public static void addNumbers(List<? super Integer>
      list, Collection<? extends Integer> source) {
    list.addAll(source);
  }
  public static <T extends Comparable<T>> T
      findMax(Collection<T> coll) {
    return coll.stream()
               .max(Comparator.naturalOrder())
               .orElse(null);
  }
  public static <T> List<T> filterByType(
      Collection<?> source, Class<T> clazz) {
    List<T> result = new ArrayList<>();
    for (Object obj : source)
      if (clazz.isInstance(obj))
        result.add(clazz.cast(obj));
    return result;
  }
}
```

PRODUCER / CONSUMER

TODO:

MISC

```java
<T extends Comparable & Collection> // multiple type bounds

Set<Integer> set = new Set<>();
Integer[] c = set.toArray(new Integer[0]);
```

GENERICS STREAM TOMFOOLERY

```java
List<? extends Media> mediaList;
public <S extends T> List<S>
  filterBySubtype(Class<S> subtype) {
    return mediaList.stream()
            .filter(subtype::isInstance)
            .map(subtype::cast)
            .collect(Collectors.toList());
}
```

ANNOTATIONS

- Metadata
- Not actually part of the code

TODO:
slides 10

- @Override
- @Test
- @Json...

DEFINING

```java
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface Author {
  String name();
  String date() default "N/A";
}

@Author(name = "UwU", date = "idon'tvalidateinput")
public class WeeOoo { }
```

TODO:
Examples Woche03

REFLECTION

- Information of Class (private and public): Methods, Attributes, Parent class, Implemented interfaces
- Calling methods possible
- Usages: Debugger, Serialization, Frameworks, Remote Procedure Call, ORM

```java
// all of these `throws SecurityException`
public Field[] getDeclaredFields()
public Method[] getDeclaredMethods()
public Constructor<?>[] getDeclaredConstructors()

System.out.println(dump(new Student("a", "b"), 0));
// {
//   firstName = a
//   lastName = b
// }
Class c = "s".getClass(); // java.lang.String
```

METHOD

```java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Init { }
// ...
public class User {
  private String needsInit;
  @Init
  private void initObj() {
    this.needsInit = "wowie";
  }
}
```

CLASS

TODO:
getDeclaredConstructor().newInstance()

```java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface JsonSerializable { }
// ...
@JsonSerializable
public class User { }
```

ATTRIBUTE

```java
@Retention(RetentionPolicy.RUNTIME)
@Target({ ElementType.FIELD })
public @interface JsonElement {
    public String key() default "";
}
// ...
public class User {
    @JsonElement
    private String firstName;
}
```

VALIDATION

```java
@Min(value = 18, message = "Age must be >= 18")
@Max(value = 99, message = "Age must be <= 99")
private int age;
@NotNull(message = "Name cannot be null")
private String name;
```

EXAMPLE

```java
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface WebController { }
// ...
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Get {
    String path();
    String contentType() default "application/json";
}
// ...
@WebController
public class Controller {
  private User user = new User("frank", "meier");
  @Get(path = "/user")
  public String user() {
    return new ObjectToJsonConverter()
                .convertToJson(user);
  }
}
// ...
public class WebFramework {
  public void addController(Class<?> controllerClass)
                        throws Exception {
  if (!controllerClass
        .isAnnotationPresent(WebController.class))
    throw new Exception("Is not a WebController");

  for (Method method : controllerClass
                        .getDeclaredMethods())
    if (method.isAnnotationPresent(Get.class)) {
      method.setAccessible(true); // best practice or sth, idk, i write code in actually good languages
      var annot = method.getAnnotation(Get.class);
      routeHandlers.put(
        annot.path(),
        new WebHandler<>(method,
                 controllerClass
                   .getDeclaredConstructor()
                   .newInstance(),
                 annot.contentType())
      );
    }
  }
}
```

ALGORITHMS

Brute-force: slow, stupid, go shame yourself
Greedy: Take best option each step (optimize locally). Fast. Not necessarily global optimum
Backtracking: Trial and error. Best overall option. Higher compute costs
Divide and conquer: Binary search
Dynamic programming: Recursion optimization? Tabulation. Iterative reuse of previous results

BACKTRACKING

TODO:
slides 87

TODO:
example

KNIGHTSTOUR

TODO:
diagram

```
boolean tour(int[][] visited, int x,int y, int pos) {
  visited[x][y] = pos;
  if (pos >= N * N) return true;
  for (int k = 0; k < 8; k++) {
    int newX = x + row[k];
    int newY = y + col[k];
    if (isValid(newX, newY)
        && visited[newX][newY] == 0
        && tour(visited, newX, newY, pos + 1))
      return true;
  }
  visited[x][y] = 0;
  return false;
}
```

BIG O NOTATION

- Worst case scenario
- Atomic operations = constant time
- Runtime measured as sum of primitive operations

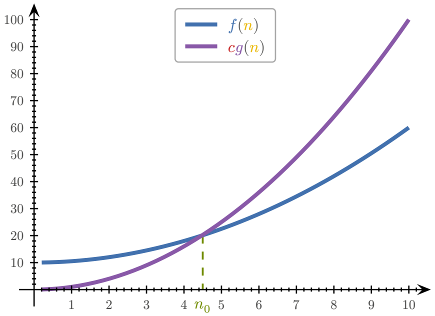
Notation

f	Algorithm	n	Size of Problem
g	Complexity Class	c	>0
n_0	≥ 1		

$$f(n) = \text{Number of steps}$$

$g =$ complexity class

TODO:
diagram $f \leq cg$, asymptotic, count primops, recursion, summary (slides 52)



$$f(n) \text{ is } O(g(n)) \text{ if } c > 0 \wedge n_0 \geq 1$$
$$f(n) \leq cg(n) \text{ for } n \geq n_0$$

f in order g:

$$f, g : \mathbb{N} \to \mathbb{N}$$
$$n_0, c \in \mathbb{N} \wedge \forall n \geq n_0 : f(n) \leq c \cdot g(n)$$
$$\Rightarrow f \in O(g) \Leftrightarrow f = O(g)$$
$$O(n) = \text{Complexity class}$$

RULES

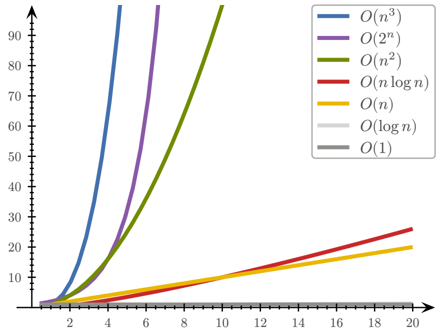
If $f(n)$ is polynomial of degree d, then $f(n) \in O(n^d)$
- ignore lower powers
- ignore constants

Use most optimal function (lowest power)
- $2n$ is $O(n)$ better than $2n$ is $O(n^2)$

Simplify as far as possible
- $3n + 5$ is $O(n)$ instead of $3n + 5$ is $O(3n)$

COMPLEXITY CLASSES

Name	Class	Example
Constant	1	Array read
Logarithmic	$\log n$	Binary search
Linear	n	Linear search
Log-linear	$n \log n$	Merge+Quick sort
Quadratic	n^2	Bubble+Insertion sort
Cubic	n^3	Solve equation
Polynomial	n^k	Various algorithms
Exponential	2^n	Calculate all subsets
Factorial	$n!$	Calculate all permutations



TODO:
Counting primitive operations (slides 55)

MAFS

$$1 + 2 + ... + n = \sum_{k=1}^{n} \qquad = \frac{n(n+1)}{2}$$
$$1 + q + q^2 + ... + q^n = \sum_{k=0}^{n} q^k \qquad = \frac{1 - q^{n+1}}{1 - q}$$
$$\overset{q=2, n=\log_2(n)}{=} \qquad 2n - 1$$

SORTING ALGORITHMS

TODO:
Sorting diagrams

BOGOSORT

The best sorting algorithm

STALINSORT

The most efficient sorting algorithm

SELECTION SORT

- Search for smallest elem in unsorted part and move in front
- Less mutations in array
- Same performance even if list almost sorted

```
public static void selectionSort(int[] a) {
  int n = a.length;
  for (int i = 0; i < n - 1; i++) {
    int minimum = i;
    for (int j = i + 1; j < n; j++)
      if (a[j] < a[minimum])
        minimum = j;
    swap(a, i, minimum);
  }
}
```

$$(n-1) + (n-2) + ... + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} \sim O(n^2)$$

INSERTION SORT

- Take elem and put into correct place in sorted part
- Good for already partially sorted arrays

```
public static void insertionSort(int[] a) {
  int n = a.length;
  for (int i = 1; i < n; i++)
    for (int j = i; j > 0 && a[j] < a[j - 1]; j--)
      swap(a, j, j - 1);
}
```

$$1 + ... + (n-1) + n = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \frac{n^2 + n}{2} \sim O(n^2)$$

BUBBLE SORT

- Compare neighboring elems and swap if needed

```
public static void bubbleSort(int[] a) {
  for (n = a.length; n > 1; n--)
    for (i = 0; i < n - 1; i++)
      if (a[i] > a[i + 1])
        swap(a, i, i + 1);
}
```

$$(n-1) + (n-2) + ... + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} \sim O(n^2)$$

COUNTING SORT

```
public static void countingSort(int[] a) {
  int k = Arrays.stream(arr).max().getAsInt();
  int[] c = new int[k + 1];
  for (int i : a) c[i] += 1;
  int pos = 0;
  for (int i = 0; i < k; i++)
    for (int j = 0; j < c[i]; j++) {
      a[pos] = i;
      pos++;
    }
}
```

$$n + n + k + n = 3n + k \sim O(n + k)$$

SHELL SORT

- Sort pairs of far away elems
- Sort the partially sorted array through insertion sort

```
public static void shellSort(int[] a) {
  int n = a.length;
  int h = 1;
  while (h < n/3) h = 3 * h + 1;
  while (h >= 1) {
    for (int i = h; i < n; i++)
      for (int j = i; j >= h && a[j] < a[j - h];
              j = j - h) swap(a, j, j - h);
    h /= 3;
  }
}
```

BINARY SEARCH

TODO:
iterative vs recursive (Woche04 Aufgabe3)

```
public static <T extends Comparable<T>> boolean bs(
    List<T> data, T target, int low, int high) {
  if (low > high) return false;
  int pivot = low + ((high - low) / 2);
  if (target.equals(data.get(pivot)))
    return true;
  else if (target.compareTo(data.get(pivot)) < 0)
    return searchBinary(data, target, low,pivot - 1);
  else
    return searchBinary(data, target, pivot+1, high);
}
```

$$O(\log n)$$

RECURSION

When a function calls itself. FP <3

LINEAR RECURSION

Recursive call starts ***at most one*** further recursive call

RECURSIVE → ITERATIVE

```
static int[] reverseArrRec(int[] a, int i, int j) {
  if (i < j) {
    int temp = a[j];
    a[j] = a[i];
    a[i] = temp;
    reverseArray(a, i + 1, j - 1);
  }
  return a;
}
static int[] reverseArrIter(int[] a, int i, int j) {
  while (i < j) {
    int temp = a[j];
    a[j] = a[i];
    a[i] = temp;
    i = i + 1;
    j = j - 1;
  }
  return a;
}
```

TAIL RECURSION

Linear recursion where the recursive call is ***last***

```
static int recsum(int x) {
  if (x == 0) return 0;
  else return x + recsum(x - 1);
  // no tail recursion because "+" is evaluated last
}
static int tailrecsum(int x, int total) {
  if (x == 0) return total;
  else return tailrecsum(x - 1, total + x);
}
```

BINARY RECURSION

All non-terminal calls have ***two*** recursive calls

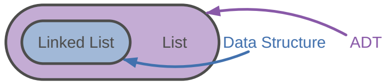
TODO:
example (slides 76,77,78)

TODO:
$O(n)$ (slides 84)

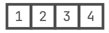
DATA STRUCTURES

TODO:
code examples

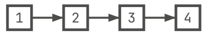
ADT: Abstract Data Type (e.g. Interface). "What", not "How"



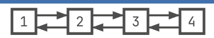
ARRAY



LINKED LIST



DOUBLY LINKED LIST



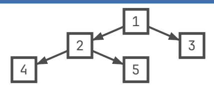
STACK

LIFO

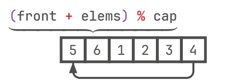
QUEUE

FIFO

TREE



RING BUFFER




```
class RingBuffer {
  private int[] buffer;
  private int size;
  private int start;
  private int end;
  private boolean isFull;

  public RingBuffer(int size) {
    if (size <= 0)
      throw new IllegalArgumentException("Size must be
greater than 0");
    this.size = size;
    this.buffer = new int[size];
    this.start = 0;
    this.end = 0;
    this.isFull = false;
  }

  public void append(int item) {
    buffer[end] = item;
    end = (end + 1) % size;
    if (isFull)
      start = (start + 1) % size;
    if (end == start)
      isFull = true;
  }

  public List<Integer> getData() {
    List<Integer> items = new ArrayList<>();
    if (!isFull) {
      for (int i = 0; i < end; i++)
        items.add(buffer[i]);
    } else {
      for (int i = start; i < size; i++)
        items.add(buffer[i]);
      for (int i = 0; i < end; i++)
        items.add(buffer[i]);
    }
    return items;
  }
}
```

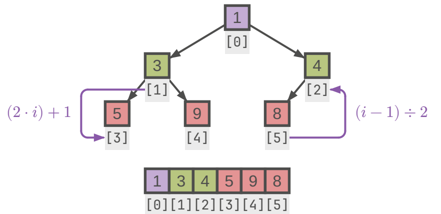
PRIORITY QUEUE

Method	Unsorted List	Sorted List
insert	$O(1)$	$O(n)$
min	$O(n)$	$O(1)$
removeMin	$O(n)$	$O(1)$

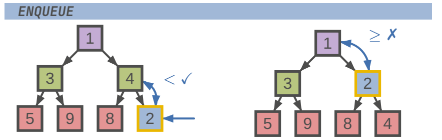
HEAP

- Binary tree
- Layers $0, ..., h-1$ fully populated, layer h filled from left
- Operations always lowest RHS so that tree stays consistent

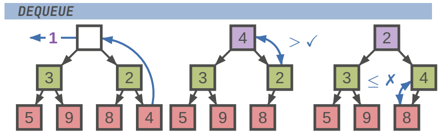
Min-Heap: Keys of nodes are **smaller** or equal than children's
Max-Heap: Keys of nodes are **larger** or equal than children's



ENQUEUE



DEQUEUE



HEAP SORT

Dequeue elems from head into list, resorting tree on each iter

TODO:

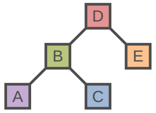
TODO:
Woche09 Aufgabe4

BINARY SEARCH TREE (BST)

- All subnodes of **left** subtree are **smaller** than root
- All subnodes of **right** subtree are **larger** than root

TODO:
Woche10 Aufgabe1

TRAVERSING



Preorder (W-L-R): D B A C E
Postorder (L-R-W): A C B E D
Inorder (L-W-R): A B C D E
Breadth-First/ Level-Order: D B E A C

TODO:

MORE DATA STRUCTURES

Set: Elements of same type, no duplicates, without order
Multiset: Set with duplicates
Map: KV-pairs with unique keys
Multimap: Key can have multiple values

HASHING

Hash-function h maps e onto $[0, n-1]$. $h(e)$ = position of e

$$e \xrightarrow{\text{hash code}} \mathbb{Z} \xrightarrow{\text{compression function}} \overbrace{[0, n-1]}^{\text{Index}}$$

Properties of good hash functions:
- Equal distribution
- Fast computation
- Constant time complexity
- Deterministic
- Incorporates as much information from key as possible

HASH FUNCTIONS

- Modulo primes
- Memory address (`Object.hashCode()` default)
- Byte/Integer cast `ByteBuffer.wrap(b).getInt()`
- Component sum (eg. sum string char codepoints)
- Polynomial accumulation $a_0 + a_1 z + a_2 z^2 + ... + a_{n-1} z^{n-1}$
  - $z = 33$ for strings
- Horner-Schema (identical, but easier to compute):
  - $a_0 + z \cdot (a_1 + z \cdot (a_2 + ... + z \cdot a_{n-1}))$

EQUALITY

It is generally necessary to override `hashCode` whenever `equals` is overridden

`x.equals(y) => x.hashCode() == y.hashCode()`

HASHING OBJECTS

```java
// Horner schema with z = 31 and start = 17
@Override
public int hashCode() {
  int result = 17;
  for (Object field : fields) result = 31 * result
      + (field == null ? 0 : field.hashCode());
  return result
}
```

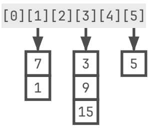
ANOMALIES

- Empty spaces
- Collisions
- Overflow

MANAGING HASH COLLISIONS

CLOSED ADDRESSING/SEPARATE CHAINING

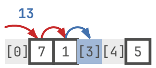
Each bucket is a list.
Search: $O(\text{entries}/\text{buckets})$



- Larger storage overhead
- Acceptable performance under high load

OPEN ADDRESSING

e "overflows" into next empty bucket
→ Must probe 2 times in example



TODO:
Problem: Primary Clustering

Linear probing: $s(k, i) = h(k) + i$
Linear probing backwards: $s(k, i) = h(k) - i$
Quadratic probing: $s(k, i) = h(k) + i^2$

TODO:
diagram, slides 10,11

TODO:
deletion (slides 16-21)
- Only mark as "deleted"
- Move next best candidate to deleted position

TODO:
access (slides 22-27)
Probability for finding free spot:

n = entries $\quad N$ = buckets

$a = \frac{n}{N}$ = probability: occupied $\quad p$ = probing steps

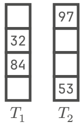
$P(X = p) = a^{p-1}(1 - a)$

$$E(X) = \sum_{x=1}^{\infty} p \cdot P(X = p) = \frac{1}{1 - a} = \text{Nr. accesses}$$

- Resize/Rehash compute intensive
- No storage overhead
- Bad performance under high load

CUCKOO-HASHING

$h_1(x) = x \bmod 5 \qquad h_2(x) = x/5 \bmod 5$



Insert: If $h_1(x)$ = null ⇒ T_1, else push y into T_2
If no cycle → $O(1)$, if rehash is needed → $O(n)$ (`MAX_CYCLE`)
Search: $O(1)$ ⇒ only has to check 2 positions
Rehashing: $x \bmod n \Rightarrow x \bmod (n + m)$

TODO:

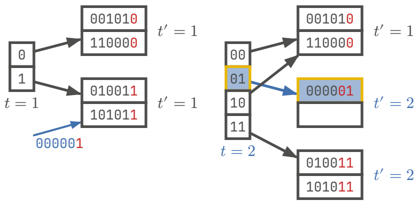
EXTENDIBLE HASHING

TODO:

- Resize/Rehash easier
- Larger storage overhead
- Good performance under high load

```java
// getting index
int idx = key.hashCode() & ((1 << globalDepth) - 1);
// when to rehash
if (bucket.getLocalDepth() == globalDepth)
  growHashTable();
else
  splitBlock();
// splitting block
int highBit = 1 << localDepth;
for (Entry e : page.entries) {
    int h = e.key.hashCode();
    Page newPage = (h & highBit) != 0 ? p1 : p0;
    newPage.put(e.key, e.value);
}
```

TODO:
slides 64



DESIGN PATTERNS

ITERATOR

```java
public interface Iterator<E> {
  boolean hasNext();
  E next() throws NoSuchElementException;
  void remove() throws UnsupportedOperationException,
IllegalStateException;
}
```

VISITOR

```java
// languages with mutability
// ...
// look at what they need just to mimic a fraction of
our power
public interface TagVisitor {
  void visit(HtmlTag html);
  void visit(HeadTag head);
  void visit(BodyTag body);
}
public class HtmlTag implements Tag {
  @Override
  public void accept(TagVisitor visitor) {
    visitor.visit(this);
    headTag.accept(visitor);
    bodyTag.accept(visitor);
    visitor.leave(this);
  }
}
```

- separate algorithms and data
- keep algorithms in one place and not distributed over objs

TEMPLATE METHOD

TODO:

EULER TOUR TRAVERSING

TODO:

COMPOSITE

TODO:
- Composite
- Client